DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics: Games Engineering

Hardware-accelerated Raytracing in Dynamic Environments using Vulkan

Christian Mulser

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics: Games Engineering

Hardware-accelerated Raytracing in Dynamic Environments using Vulkan

Hardwarebeschleunigtes Raytracing in dynamischen Umgebungen mittles Vulkan

Author:	Christian Mulser
Supervisor:	Prof. Dr. Rüdiger Westermann
Advisor:	Prof. Dr. Rüdiger Westermann
Submission Date:	September 22, 2025

I confirm that this bachelor's thesis in informatics: games engineering is my own work and I have documented all sources and material used.

Munich, September 22, 2025 Christian Mulser

Acknowledgments

I thank my supervisor, Rüdiger Westermann, for guiding me along the way and providing useful feedback to my work. I also thank Sebastian Wohner for providing access to the required modern RTX-GPUs and for quickly helping me out with technical difficulties.

Abstract

Contents

1. Introduction

2. Related Work

3. Technologies

3.1. Operating System

Ubuntu 24.04.2 LTS

3.2. Build Tools

- Premake 5
- GNU Make 4.3
- Visual Studio 2022

The project uses Premake 5, which does not build the project directly, but it is a command line utility which reads a scripted definition of a software project and, most commonly, uses it to generate project files for toolsets like Visual Studio, Xcode, or GNU Make [ric23]. It is a simpler alternative to CMake. On Linux we use it to create “Makefiles”, which then ultimately build our project. On windows we create Solution- and Project-files for Visual Studio 2022 and we can build and run the project inside Visual Studio.

3.3. Graphics API

As the graphics API for our project we chose **Vulkan 1.2**. Vulkan is a modern platform-agnostic API to handle GPU operations. It is the successor to OpenGL, which was discontinued in 2016. Vulkan allows for much more fine-grained control over the GPU, supports multi-threading and for this project most importantly raytracing, through extensions like `VK_KHR_ray_tracing_pipeline` and `VK_KHR_acceleration_structure`. Since these extensions were introduced in Vulkan 1.2, that is the version used in the project.

3.4. Programming Language

At the start of the project Rust was considered as the projects programming language. Rust is a new, memory-safe, low-level programming language and a modern alternative to languages like C and C++. While Rust does have a lot of advantages over C++, we chose **C++** in the end. The main reason for this is that the Vulkan API was designed for C/C++. There are a lot of great Vulkan-libraries for Rust, like the popular "Ash" create, which is generated from

"vk.xml". However, way you use the API does differ from the classic C API. This could be problematic, considering that nearly all the few resources relating to raytracing in Vulkan use C/C++. This could have made it difficult to translate the raytracing code into Rust later down the line, so C++ was the safer choice.

3.5. 3D Scene File Format

The dynamic 3D scenes, that the project uses, are stored in the **glTF-2.0** file format. It stores its models in a scene graph and supports skeletal animation as well as morph animations. By default, it stores the scene data in the popular JSON format. Additionally, buffers are stored in binary in BIN files and textures are stored in image files (PNG, JPEG, ...). Since the file format was developed by Khronos Group, which is also behind Vulkan, the data inside the buffers is laid out in a way that makes it easy to use in Vulkan.

3.6. Libraries

3.6.1. Volk

volk is a meta-loader for Vulkan. It allows you to dynamically load entry points required to use Vulkan without linking to vulkan-1.dll or statically linking Vulkan loader. Additionally, volk simplifies the use of Vulkan extensions by automatically loading all associated entry points [https://github.com/zeux/volk].

3.6.2. VMA (Vulkan Memory Allocator)

Memory Management in Vulkan is quite difficult. For example, when you create a buffer or image in Vulkan there is no memory associated with it. The programmer needs to manually allocate block of memory with a certain type from a certain memory heap. The properties of these memory heaps depend on the underlying physical device and need to be queried first. Additionally, creating a new memory allocation for each resource may be suboptimal, since the number of allocations can be very limited and many allocations may negatively impact performance. VMA (Vulkan Memory Allocator) simplifies, automates and optimizes memory management in Vulkan.

3.6.3. vk-bootstrap

Initializing Vulkan involves a lot of boilerplate code, which is going to be basically the same in every Vulkan application. VkBootstrap can be used to reduce the amount of boilerplate code in a project.

This library simplifies the tedious process of:

- Instance creation

- Physical Device selection
- Device creation
- Getting queues
- Swapchain creation

[Lun25]

3.6.4. cgltf

To load a glTF file in the project, cgltf is used. cgltf parses the glTF file, which is stored in JSON. It would also have been possible to read the JSON directly, but cgltf takes care of validating the file, loads the required buffers from their BIN-files into memory and creates a C-structure, which is a lot more comfortable to use. Our project does not this structure directly, but rather used to generate a `Scene` object.

3.6.5. CLI11

CLI11 is a header-only library written in modern C++, that is designed to parse the command-line options of your application.It is simple, feature-rich and safe. CLI11 allows you to define flags, options, commands and subcommands. Your can also add certain restrictions to your parameters, like only allowing positive numbers or that a parameter is an existing path.

4. Implementation

4.1. Project Setup

The projects as well as this thesis' source code are avaibable in the following Github repository:
`https://github.com/mulsicpp/bachelor_thesis_2025.git`
The project is organized in the following directories:

- "assets": This folder contains the projects' assets. It is further divided into scenes, which contains the glTF test scenes, and "shaders", which contains the shaders, that are used in the different pipelines.
- "external": This folder contains all of the "complex""" external libraries. "Complex" in this context means, that the library has multiple header or source files or it is in binary form.
- "premake": This folder contains the premake executables for both windows and linux.
- "src": This folder contains all of our source files for the project. It is described in more detail in the next chapter.

Additionally, there are some scripts, that are used to build the project.

4.1.1. Source Code Organization

In this section we will specifically look at how the "src" folder is organised.

Directly in the "src" folder we find "main.cpp", which contains our entry point of the application. The "external" folder once again contains external libraries, but this time the libraries, which are in the form of a single header file. These usually need to be included in a C/C++ file and need to be compiled. That's why it makes sense to keep these libraries with our other source files. The "utils" folder contains utilities, that may be useful all throughout the project. It includes functionalities like debug logging, move semantics and file loading. All the folders with a "vk_" prefix contain code, which abstracts the low-level Vulkan API code. The goal of these abstractions is, that the low-level code should only ever be used in these folders. The rest of the project should only use the abstracted API. Each of the folders focuses on a different area of Vulkan:

- "vk_core": The most important class in this folder is the `Context` class. It is designed as a singleton, which contains the `VkInstance`, `VkPhysicalDevice`, `VkDevice`, `VkQueues` and `VmaAllocator`. After the context gets created it can be accessed anywhere. The

folder also contains the `Handle<T>` class, which ensures that a handle only has one owner, the `CommandBuffer` abstraction and functionality to deals with `VkFormat`.

- "vk_resources": This folder contains Vulkans resource objects like `Buffer` and `Image`, as well as `ImageView` and `SubBuffer`.
- "vk_pipeline": This folder contains Vulkan objects, that are related to the setup of a pipeline. This includes `Pipeline`, `PipelineLayout`, `Shader`, `RenderPass`, `Framebuffer` and `DescriptorPool`.
- "vk_sync": Contains the vulkan synchronization objects `Fence` and `Semaphore`.
- "scene": This folder contains the `Scene` class, which describes a dynamic scene with a scene graph. All the other classes used by or contained in the scene, like `Mesh`, `Node`, `Camera` and `Animation`, can also be found in this folder.
- "rendering": This folder contains 3 different renderers:
  - `Rasterizer`: Render a scene from the viewpoint of a camera into a framebuffer using traditional rasterisation.
  - `Raytracer`: Render a scene from the viewpoint of a camera into an image using modern hardware-accelerated raytracing.
  - `Skinner`: Calculates the skinned positions of a mesh during an animation.

4.2. Vulkan Abstraction

4.2.1. Move Semantics

As already mentioned in the Methods and Technology chapter, Rust was originally considered as programming language, because it is modern and memory safe. One of Rust's core features is ownership, which means that each value/object can only be owned and managed by only one variable, and when that variable goes out of scope, it alone is responsible for cleaning up the object. When that variable gets assigned to another variable, the new variable becomes the new owner (i.e. the value gets moved). Ownership would be a very nice feature to manage Vulkan objects. Vulkan objects should not be copied on assignment but only moved and when the object is no longer needed (i.e. the owner goes out of scope), it should automatically be destroyed. While C++ does not support ownership directly, it does have so-called move semantics, with which we can emulate ownership. The ownership of Vulkan handles is managed by the `Handle<T>` class. To do this the class declares a default constructor, move constructor and destructor and overrides the move assignment operator. It is also important, that the copy constructor and copy assignment operator are deleted. The reason for that is that copying a Vulkan object is usually very expensive or not even possible and it is almost never necessary. When we want to copy an object, we need to explicitly create a new one first. This avoids unnecessary and complicated accidental creation and copying of new objects.

4.2.2. Shared Pointers

Since C++11 the language supports so-called smart pointers, which make memory management safer and less complicated. One of these smart pointers is `std::shared_ptr<T>` (or our alias `ptr::Shared<T>`). This type of pointer allocates an object, which can then be shared between multiple owners. The object lives while it has at least one owner, and the last object to go out of scope cleans up the object. This structure can be combined very nicely with our previously defined handle. A Vulkan object can only have one owner, which is a harsh limitation and often causes problems. If we wrap the object in a shared pointer, it can be shared across multiple owners. We also added the class `ToShared<T>`, which a class can inherit from. An instance of that class can then be moved into a shared pointer by calling `to_shared()`.

4.2.3. The Builder Pattern

As mentioned, before we want the creation of Vulkan objects to be very explicit. The first method to create objects in C++ you think of are constructors, but they prove to be problematic for a couple of reasons. The first problem is that constructors are often called implicitly (e.g. the default constructor gets called when declaring a variable). Default construction only makes sense for very few Vulkan objects (like for example a VkFence) and we do not want a Vulkan object to be created when implicitly (or explicitly) calling the default constructor. So, for that reason, the default constructor simply does not create a Vulkan object or, in other words, leaves the handle at `VK_NULL_HANDLE`. Now if a different constructor actually creates an object, the behavior of constructors becomes inconsistent regarding Vulkan object creation. In addition to that, Vulkan objects are almost always created using a `*CreateInfo` struct, which may take a lot of parameters. To solve all of these issues we implemented a `Builder` class for most Vulkan objects. Some exceptions are `Fence` and `ImageView`, which are created with `Fence::create(...)` and `ImageView::create_from(...)` respectively. This makes it very clear when a Vulkan object is created, since it can only be created using a `Builder` or a static method (like `Fence::create(...)`) and especially it can never be created by a constructor. This is also the reason Vulkan objects never have constructor, besides the default constructor.

```
vk::Buffer buffer; // declare a buffer (handle is VK_NULL_HANDLE)

buffer = vk::BufferBuilder()
    .add_usage(VK_BUFFER_USAGE_TRANSFER_DST_BIT)
    .add_usage(VK_BUFFER_USAGE_VERTEX_BUFFER_BIT)
    .memory_usage(VMA_MEMORY_USAGE_GPU_ONLY)
    .size(128)
    .build();
```

4.2.4. Basic Vulkan Object Structure

All of the Vulkan objects follow a basic structure, which can be seen in the following code sample:

```
class Foo : public utils::Move, public ptr::ToShared<Foo> {
    friend class FooBuilder;
private:
    Handle<VkFoo> foo_handle{};
    // Additional member variables

public:
    Foo() = default;

    inline VkFoo handle() const { return foo_handle; }
    // Additional getter functions

    // Additional functions
};

class FooBuilder {
public:
    using Ref = FooBuilder&;
private:
    // build parameters, e.g. VkFormat _format;

public:
    FooBuilder() = default;

    // parameter setter functions, e.g. Ref format(VkFormat format);

    Foo build() const;
}
```

Figure 4.1.: An example of how to use command buffers in your code.

It is important to note, that `Foo` is just a placeholder for an actual Vulkan object (like `Buffer`, `Image` or `Pipeline`). Usually, the name of the class is just the name of the Vulkan handle without the "Vk" prefix (e.g. `VkBuffer` $\Longrightarrow$ `Buffer`), but there are some exceptions. For example a `Shader` class holds a `VkShaderModule` handle.

The class inherits from two other classes: `utils::Move` and `ptr::ToShared<Foo>`. The `utils::Move` class implements the default move constructor and move assignment, and it

deletes the copy constructor and copy assignment. The `ptr::ToShared<Foo>` class contains the function `ptr::Shared<Foo> to_shared()`, which is just syntax sugar for moving the object into a shared pointer.

The additional variables in the class are often used to store meta-data (e.g the format and extent of an image) or to keep other objects alive in a shared pointer, that are referenced by the underlying object (e.g. the render pass of a pipeline). There usually is also an getter function for these variables.

The additional functions include functions, that actually use or manipulate the underlying object. Often, these functions have a "cmd_" prefix, which means that they are wrappers around one or multiple commands (e.g. `Buffer::cmd_copy_into`).

Even though not every Vulkan object has uses a builder class, it is also shown in the sample code, since most Vulkan objects use it.

4.2.5. The Vulkan Context

Before we can actually do any work in Vulkan, we need to do a lot of setup. This usually includes the following steps:

1. Create a `VkInstance`.
2. Select a `VkPhysicalDevice`, which supports the extenstions, features and capabilities required by the project.
3. Create a `VkDevice` from the selected `VkPhysicalDevice`.
4. When creating the `VkDevice`, `VkQueue`s from a certain queue family need to be enabled, to be able to submit command buffers later.

These Vulkan objects are accessed very frequently throughout the project. For example, you need to pass the `VkDevice` as a parameter when creating or destroying most Vulkan objects and a command buffer, which executes a sequence of Vulkan commands, needs to be submitted to a certain `VkQueue`. Keeping track of all of these Vulkan objectsis quite tideous, which is why we introduced the Vulkan context. It is represented by the `vk::Context` class, which stores these Vulkan objects, allows the programmer to access them anywhere and reduces the effort of setting up Vulkan.

The Vulkan context contains the following objects:

- the `VkInstance`
- the `VkPhysicalDevice` and `VkDevice`
- an optional `GLFWwindow*` from which a `VkSurfaceKHR` is created and also stored (this is necessary for rendering to a window, but it will not be used in our main application, since it is headless)
- the `vk::CommandManager`

- the `VmaAllocator`

The class is designed similar to a singleton, but it deviates in some parts. A singleton usually creates its instance at the start of the application or when it is used for the first time (lazy instantiation). However, `vk::Context` can not be implicitly instanciated this way, because it needs some parameters stored in a `vk::ContextInfo`, when being instanciated. For this reason we need to explicitly create the Vulkan context using `vk::Context::create(...)` with the desired `vk::ContextInfo` before calling `vk::Context::instance()` for the first time or we will get a `nullptr`. While all other objects in our project get cleaned up automatically when going out of scope, the Vulkan context also needs to be destroyed explicitly using `vk::Context::destroy()` when it is no longer needed. If we now want to get the current `VkDevice`, we can call `vk::Context::instance()->get_device()`.

4.2.6. The Command Manager

Queue management in Vulkan is a big and complex topic. Vulkan exposes the different queue families on your GPU, which support a subset of the following command types:

- Graphics
- Transfer
- Compute
- Present

When submitting a command buffer to a queue, the queue needs to belong to a queue family, which supports the different types of the commands in the command buffer.

The management of Vulkan queues is completly up to the programmer. You can query the capabilities of the queue families and then enable an arbitrary amount of queues from different queue families. This is especially useful in a multi-threaded application, where you can submit two command buffers to two different queues and they will be executed in parallel.

The `vk::CommandManager` class makes queue management easier. It manages the queue families, aswell as their enabled queues and created descriptor pools. Our application is signle-threaded in order to avoid too much complexity and because our main objective is to benchmark the performace of dynamic raytracing, which works better in a single-threaded environment. For this reason we also do not need multiple queues or queues designated for a certain type of operations (for example a designated transfer queue). However we do need a queue for every command type, which our command manager ensures. For every command type, it will iterate over all queue families and select the first family, which supports the command type. When creating the device, we enable the first queue of every queue family, which has been selected at least one time for a command type.

The following are some examples of possible queue setups:

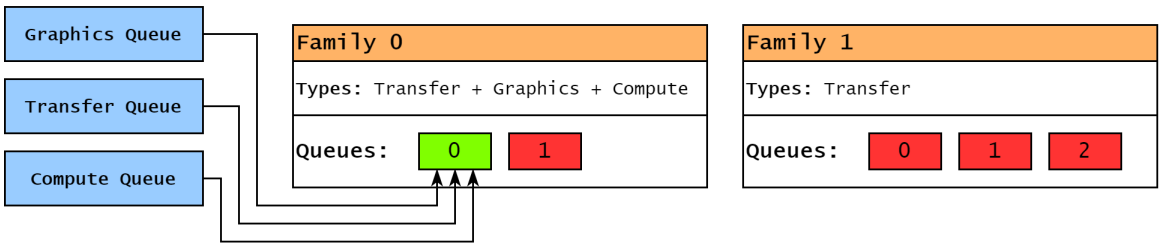

Figure 4.2.: Queue setup example with only one queue

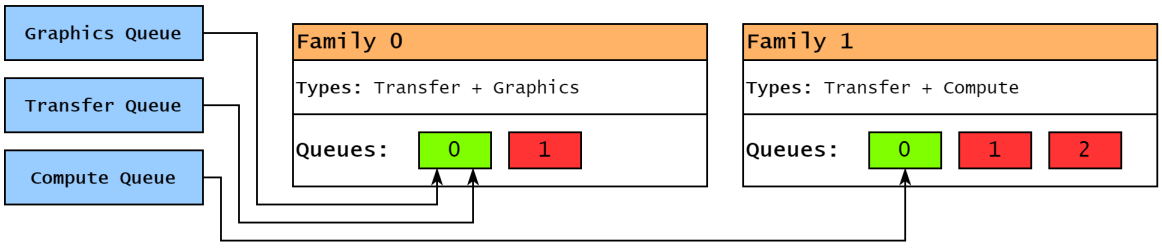

Figure 4.3.: Queue setup example with two queues

We also need a command pool to be able to allocate command buffers. When creating a command pool, a queue family has to be specified. Because the allocated command buffers can only be submitted to queues of the specified family, we create one command pool for every queue family that is used.

4.2.7. The VMA Allocator

As already mentioned in the chapter Technologies (subsection 3.6.2), our project uses VMA (Vulkan Memory Allocator) to make memory management in Vulkan easier. In order to be able to be allocate memory for our buffers and images later, we need an allocator. The allocator should be easily accessible when we create a buffer or image and for that reason we will also create and store the VMA allocator in the Vulkan context. We will use this allocator later, when allocation our resource objects (see section 4.3).

4.2.8. Command Buffers

As we know, commands can not be executed in Vulkan directly, like they can for example in OpenGL. Instead, you first need to allocate a command buffer from a command pool. You can then record all of the commands you want to execute into the buffer and can then submit it for execution to a queue, that supports the type of the commands. It is important to note that when you submit the command buffer the current CPU-thread does not wait for all the commands to finish. This allows for the CPU to keep working, while the GPU runs our

commands. If we want to wait for the command buffer to finish, we can specify a fence when submitting the command buffer, which will be signaled once all commands have finished.

We abstract this logic in the `vk::CommandBuffer` class. It contains a `VkCommandBuffer` handle, a `vk::Queue`, which tells the command buffer, where it should be submitted to, and a `vk::Fence` (see subsection 4.3.1), which is used to wait for the command buffer to finish executing.

The `vk::CommandBufferBuilder` has the following two parameters:

- `queue_type`: A value of type `vk::QueueType`, which contains the elements `Transfer`, `Graphics`, `Compute` and `Present`. It is a mandatory parameter, which tells the builder to which of the queues inside the queue manager the command buffer is going to be submitted to (for example `vk::QueueType::Graphics` means the command buffer will be submitted to the graphics queue of the command manager).
- `single_use`: An optional boolean value, which is set to `false` by default. If set to `true`, it sets the `VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT` flag when recording the command buffer. This flag specifies, that the command buffer will only be submitted once, which lets the driver apply optimizations.

Inside `vk::CommandBuffer` we can find the following member functions:

- `Ref record(vk::CommandRecorder)`:

  This function takes in a `vk::CommandRecorder` as argument, which is an alias for `std::function<void(vk::ReadyCommandBuffer)>`. In other words it is a function pointer or lambda function, which takes a `vk::ReadyCommandBuffer` as an argument and returns `void`. A `vk::ReadyCommandBuffer` is simply a wrapper around a `VkCommandBuffer` handle, which can only be constructed by the command buffer. The job of the `vk::CommandRecorder` is, to record commands into the command buffer.

  Functions, that record a commands can be recognized by their `"cmd_"` prefix. These functions have to be called inside a `vk::CommandRecorder`, because thier first argument is always a `vk::ReadyCommandBuffer`.

- `Ref submit(vk::SubmitInfo)`:

  This function submits the command buffer to the queue, for which it was created for. The `vk::SubmitInfo` parameter is optional and empty by default. This parameter allows us to specify semaphores, that we want to wait for before executing the command buffer, and semaphores, that we want to signal once the command buffer is finished executing (see subsection 4.3.1).

- `Ref wait()`:

  This functions simply waits for the execution of the command buffer to finish using a fence. If the command buffer is not currently getting executed, the function does nothing.

It is also important to mention, that this function also gets called in the desctructor of the `vk::CommandBuffer`, since it is not allowed to destroy a `VkCommandBuffer` while it is in execution. This also means, that a command buffer implicitly wait for execution to finish, when it goes out of scope.

The `vk::CommandBuffer` class also contains the following static method:

`static void single_time_submit(vk::QueueType, vk::CommandRecorder)`

It creates a single-use command buffer for the queue with the specified `vk::QueueType`. Then, it records the given `vk::CommandRecorder`, submits it with an empty `vk::SubmitInfo` and waits for execution to finish. It is basically just a quality-of-life function for creating and executing single-use command buffers.

The folloing code snippet shows, how the command buffer and its functions can be used:

```
// create the command buffer
vk::CommandBuffer cmd_buf = vk::CommandBufferBuilder()
    .queue_type(vk::QueueType::Graphics)
    .single_use(true)
    .build();

// create a callback, which records the commands
vk::CommandRecorder recorder = [&] (vk::ReadyCommandBuffer ready_buf) {
    // record commands
    // ...
}

// record the commands, submit to queue and
// wait for execution to finish in one line
cmd_buf.record(recorder).submit().wait();
```

Figure 4.4.: An example of how to use command buffers in your code.

4.3. Resource Objects

Resource objects are used to store data on the GPU, which can then be read and written to in shaders. In Vulkan we distinguish two types of resource objects:

- **Buffer:**

  Buffers represent linear arrays of data which are used for various purposes by binding them to a graphics or compute pipeline via descriptor sets or certain commands, or by directly specifying them as parameters to certain commands [Khr25, see Section 12.1].

The data is raw and not associated with any format, layout or dimensionality, therefore very flexible.

Common usages:

- Vertex buffer
- Index buffer
- Uniform buffer
- Storage buffer

- **Image:**

  Images are specialized resources that have multi-dimensional access rather than the typical linear access to memory [Khr25, see Section 13]. They can also be multi-dimensional and are associated with a format and layout.

  Common usages:

  - Texture
  - Render target

We represent these two objects by the `vk::Buffer` class and `vk::Image` class. Since they are both resource objects, they are very similar in certain aspects. One challange, that they both share, is memory management, which can be quite difficult.

When creating a resource object, they are not actually backed by memory immediately, but you need to allocate a block of device memory and bind that memory to the resource. When allocating the memory, you need to specify a memory type, which corresponds to a certain memory heap and can have a set of properties, which impact the visibility of your memory aswell as performance. It is also important to mention, that too many allocations are not just bad for the performance of your application, but there is also a limit of allocations, that cannot be superceeded.

To make our lives a little bit easier, we use VMA (Vulkan Memory Allocator) (see subsection 3.6.2).

4.3.1. Synchronization Objects

A lot of vulkan operations can be executed in parallel, which is why we need **synchronization objects**. There are two types:

- **Semaphore:**

  A semaphore is used to synchronize two operations, which both happen on the GPU. We do this by specifying and array of wait-semaphores and an array of signal-semaphores. The GPU operation waits for all wait-semaphores to be signaled before starting and signals all signal-semaphores when it is finished.

- **Fence:**

  A fence is used to synchronize operations on the GPU with the CPU. This is done by specifying a fence when starting a GPU operation. When the GPU operation finishes, the fence gets signaled. The CPU can wait on a fence, which will block the thread until the fence gets signaled.

The `vk::Semaphore` class encapsualtes a semaphore. It only holds a `Handle<VkSemaphore>`, that can be accessed by the member function `handle`. The `vk::Semaphore` class does not have any other functionality, because it does not need it. A semaphore only ever gets passed as a parameter to GPU operations and these GPU operations signal and unsignal the semaphore. You never use the semaphore explicitly, but it is used implicitly by GPU operations.

The `vk::Fence` class

4.4. Scene Graph

5. Results

6. Discussion

7. Conclusion

A. General Addenda

If there are several additions you want to add, but they do not fit into the thesis itself, they belong here.

A.1. Detailed Addition

Even sections are possible, but usually only used for several elements in, e.g. tables, images, etc.

B. Figures

B.1. Example 1

✓

B.2. Example 2

✗

List of Figures

List of Tables

Bibliography

[Khr25] Khronos Group. *Vulkan® 1.4.327 - A Specification (with all registered Vulkan extensions)*. Accessed: 2025-09-19. The Khronos Group Inc. 2025. URL: https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html.

[Lun25] C. Lunarg. *vk-bootstrap: Vulkan Bootstrapping Library*. accessed: 2025-09-19. 2025. URL: https://github.com/charles-lunarg/vk-bootstrap.

[ric23] ricecookie. *What is Premake?* boh. 2023.